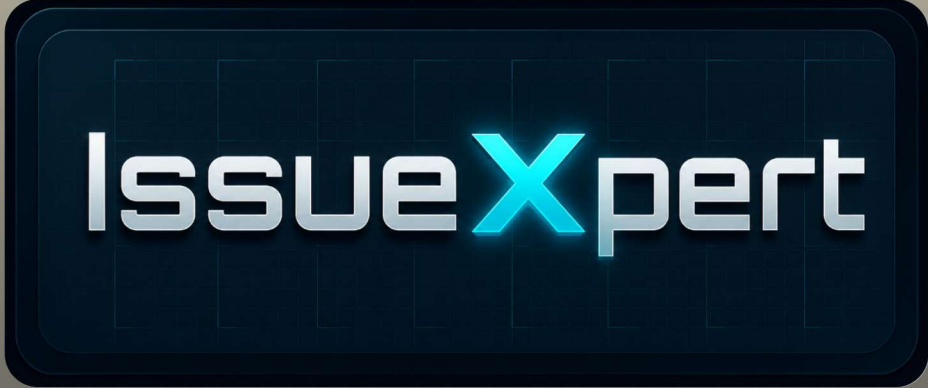


Memoria proyecto:



IssueXpert

C.I.F.P. "JUAN DE COLONIA"

Departamento de Informática

2º DAW

Autor:

Héctor Manuel Martínez Delgado

Tutor:

Ana Rosa Álvarez Rojo

Fecha:

09/0/2026

Contents

1.	Introducción.....	5
1.1.	Descripción.....	5
1.2.	Justificación.....	5
2.	Planificación	6
2.1.	Requisitos.....	6
2.1.1.	Requisitos funcionales.....	6
2.1.2.	Requisitos no funcionales.....	7
2.2.	Recursos	8
2.2.1.	Recursos hardware	8
2.2.2.	Recursos software	8
2.3.	Planificación temporal	8
2.4.	Planificación económica	9
3.	Tecnologías	10
4.	Desarrollo y secuenciación temporal	11
4.1.	Diseño	11
4.1.1.	Diagrama E/R (entidad/relación).....	11
4.1.2.	Modelo relacional.....	12
4.1.3.	Patrones de diseño del software.....	13
4.1.4.	Diseño de interfaces	19
4.1.5.	Arquitectura de frontend basada en componentes.....	24
4.1.6.	Despliegue de la aplicación	24
4.2.	Pruebas	26
4.2.1.	Pruebas funcionales	26
4.2.2.	Pruebas no funcionales	27
5.	Conclusiones finales.....	28
5.1.	Grado de cumplimiento de los requisitos fijados.....	28
5.2.	Propuestas de mejora o ampliaciones futuras	28
6.	Guías	29
6.1.	Guía de administrador	29
6.2.	Guía de usuario	32

6.2.1.	Acceso a la aplicación	32
6.2.2.	Panel principal (Dashboard)	32
6.2.3.	Gestión de tickets	34
6.2.4.	Asignación y seguimiento.....	35
6.2.5.	Perfil de usuario.....	36
7.	Referencias.....	37

Índice de figuras

Fig. 1-1-Pantalla principal de la aplicación	5
Fig. 4-1-Diagrama E/R.....	10
Fig. 4-2-Modelo relacional.....	12
Fig. 4-3-Mapa del sitio	19
Fig. 4-4-Pantalla de login	21
Fig. 4-5-Pantalla del dashboard	21
Fig. 4-6-Pantalla del panel de administrador	22
Fig. 4-7-Pantalla del modal (CRUD)	22
Fig. 4-8-Estado del backend de IssueXpert en PM2	24
Fig. 4-9-Configuración del tunel de Cloudflare	24
Fig. 4-10-Configuración de Nginx para IssueXpert (reverse proxy)	24
Fig. 6-1-Pestaña de administración de usuarios	28
Fig. 6-2-Botones de acción (eliminación, edición y creación)	28
Fig. 6-3-Modal de edición de usuario	29
Fig. 6-4-Modal de creación de usuario	29
Fig. 6-5-Perfil de usuario	30
Fig. 6-6-Pantalla de inicio de sesión	31
Fig. 6-7-Diferentes vistas de los tickets	32
Fig. 6-8-Barra de filtros	32
Fig. 6-9-Botones de acción sobre tickets (edición, eliminación y creación)	33
Fig. 6-10-Modales de creación, edición y eliminación de ticket	33
Fig. 6-11-Ticket asignado y desasignado	34
Fig. 6-12-Información de usuario con el panel de acciones de usuario	35
Fig. 6-13-Modal de cambio de contraseña	35

Índice de tablas

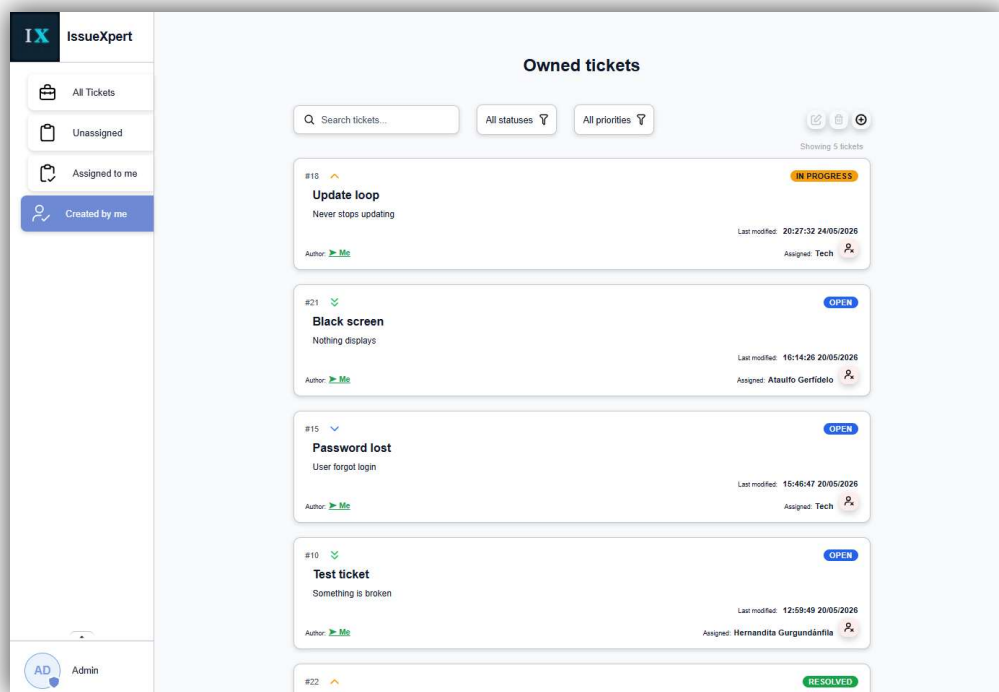
Tabla 2-1: Requisitos funcionales	6
Tabla 2-2: Requisitos no funcionales	8
Tabla 2-3: Planificación temporal del proyecto	8
Tabla 2-4: Presupuesto del proyecto	9
Tabla 3-1: Tecnologías usadas	9
Tabla 4-1: Roles y permisos	16
Tabla 4-2: Pruebas funcionales	25
Tabla 4-3: Pruebas no funcionales	26

1. Introducción

1.1. Descripción

Este proyecto consiste en la creación de una aplicación web para gestionar de incidencias. Está orientada a facilitar la comunicación entre usuarios y técnicos en una empresa u organización. Permite crear, administrar y hacer un seguimiento de cada ticket, incorporando roles diferenciados a cada usuario, mensajería y evaluación de satisfacción del usuario.

Fig. 1-1-Pantalla principal de la aplicación



1.2. Justificación

En varias situaciones laborales he experimentado lo arduo que puede llegar a ser la burocracia vinculada a la resolución de los problemas o impedimentos más cotidianos. De estas experiencias nace esta idea con un simple objetivo: facilitar estas tareas lo máximo posible.

2. Planificación

2.1. Requisitos

2.1.1. Requisitos funcionales

A continuación, se presentan los requisitos que la aplicación debe cumplir. Estos requisitos describen lo que la aplicación puede hacer para los diferentes tipos de usuarios.

Tabla 2-1: Requisitos funcionales

ID	Requisito funcional
RF01	El sistema permitirá el inicio de sesión mediante usuario y contraseña.
RF02	El sistema permitirá la gestión de usuarios (CRUD) por parte del administrador.
RF03	El sistema permitirá la asignación y gestión de roles (administrador, técnico y usuario).
RF04	El sistema permitirá la creación de tickets de incidencia por cualquier usuario autenticado.
RF05	El sistema permitirá la modificación de tickets según los permisos y el rol del usuario autenticado.
RF06	El sistema permitirá la eliminación lógica de tickets por los autores de los tickets o administradores.
RF07	El sistema permitirá cambiar el estado de los tickets (abierto, en progreso, resuelto y cerrado) por técnicos o administradores.
RF08	El sistema permitirá la asignación y desasignación de tickets al propio usuario autenticado sin son técnicos o administradores.
RF09	El sistema permitirá consultar tickets según el rol del usuario autenticado.
RF10	El sistema permitirá filtrar tickets por texto, estado y prioridad.
RF11	El sistema permitirá la gestión de categorías (CRUD) por parte del administrador.
RF12	El sistema permitirá modificar la contraseña del usuario autenticado.
RF13	El sistema validará los permisos de acceso a cada funcionalidad según el rol del usuario.

2.1.2. Requisitos no funcionales

A continuación, se presentan los requisitos no funcionales del sistema. Estos definen la calidad, el rendimiento y las restricciones de la aplicación.

Tabla 2-2: Requisitos no funcionales

ID	Requisito no funcional
RNF01	El sistema será accesible desde dispositivos de escritorio, tablets y móviles mediante diseño responsive.
RNF02	El sistema será compatible con los navegadores web más utilizados (Chrome, Brave y similares).
RNF03	El sistema garantizará la autenticación segura de usuarios mediante cifrado de contraseñas.
RNF04	El sistema mantendrá la sesión del usuario mediante autenticación basada en JWT.
RNF05	El sistema permitirá el acceso concurrente de múltiples usuarios sin pérdida de rendimiento significativa.
RNF06	El sistema ofrecerá tiempos de respuesta adecuados, manteniendo un rendimiento óptimo en la carga de vistas y datos.

2.2. Recursos

2.2.1. Recursos hardware

- ✓ Ordenador u otro dispositivo con navegador
- ✓ Conexión a internet

2.2.2. Recursos software

- ✓ Visual Studio Code (editor de código)
- ✓ Node.js (entorno de ejecución JS para el servidor)
- ✓ MySQL (SGBD que gestiona el almacenamiento de datos)
- ✓ Git / Github (control de versiones y repositorio)
- ✓ Navegadores: Brave / Chrome (interacción con la app y uso de herramientas de desarrollo)
- ✓ Postman (herramienta de pruebas con la API)

2.3. Planificación temporal

Tabla 2-3: Planificación temporal del proyecto

Descripción de la tarea	Nº de horas
Análisis	10
Diseño de la base de datos	15
Backend	60
Frontend	70
Testing	25
Documentación	20
Total horas	200

2.4. Planificación económica

Tabla 2-4- Presupuesto del proyecto

Descripción de la tarea	Precio (€)
Hosting de la aplicación (Raspberry Pi5)	35 €
Dominio (hnezado.dev)	12 €
Conectividad (internet + luz)	60 €
Hardware (Raspberry Pi 5 + SSD, amortizado a 3 años)	40 €
Subtotal infraestructura	147 €

* El coste de infraestructura se calcula como estimación anual de los recursos utilizados.

Desarrollo (200 horas * 12 € / h)	2400 €
Total (€)	2547 €

3. Tecnologías

Tabla 3-1- Tecnologías usadas

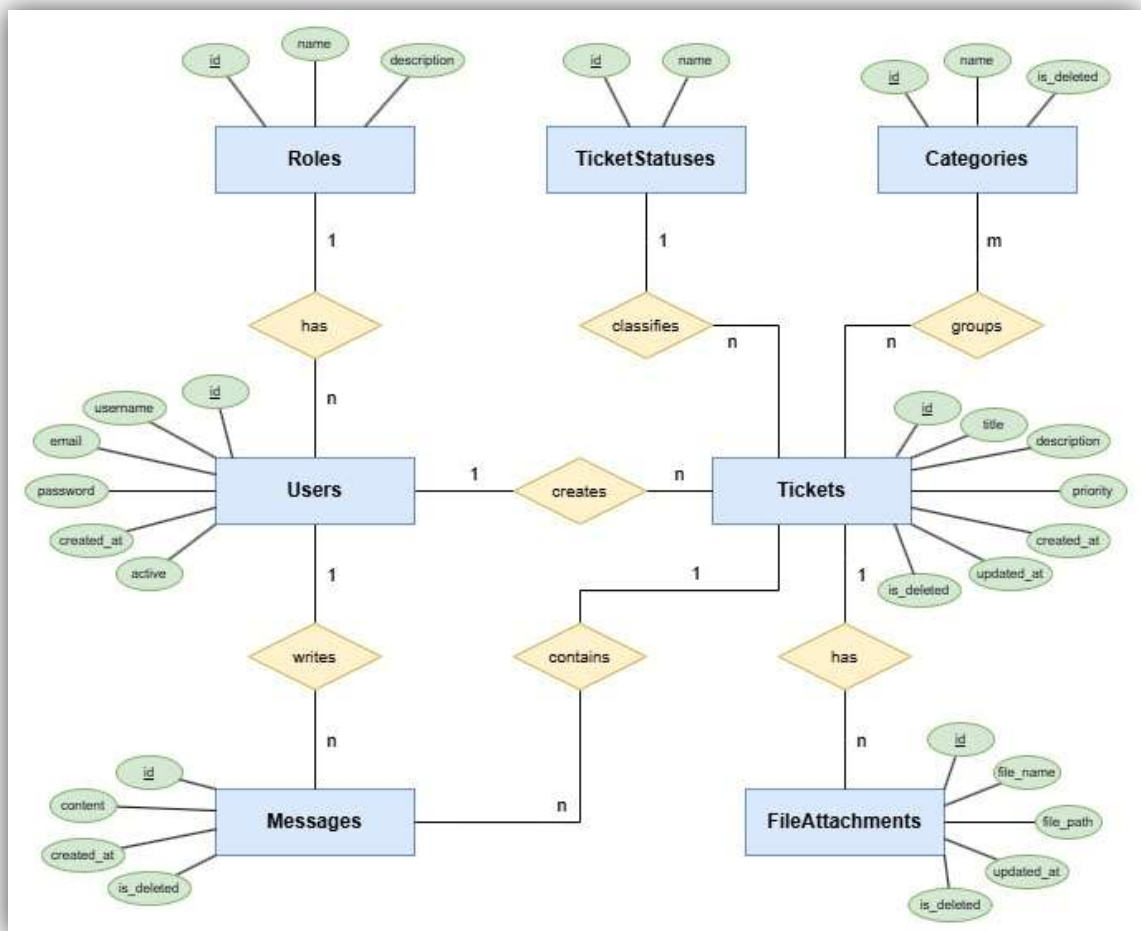
Tecnología	Descripción
HTML	Definición de la estructura y el contenido de las páginas web.
Sass	Extensión de CSS con variables, anidación, mixins y módulos para la gestión de estilos y diseño responsive.
JavaScript	Lenguaje usado para la interactividad, manipulación dinámica del DOM y lógica en el cliente.
Node.js / Express	Entorno de ejecución de JavaScript en el servidor y framework para la creación de la API.
MySQL	SGBD relacional para el almacenamiento y gestión estructurada de datos.

4. Desarrollo y secuenciación temporal

4.1. Diseño

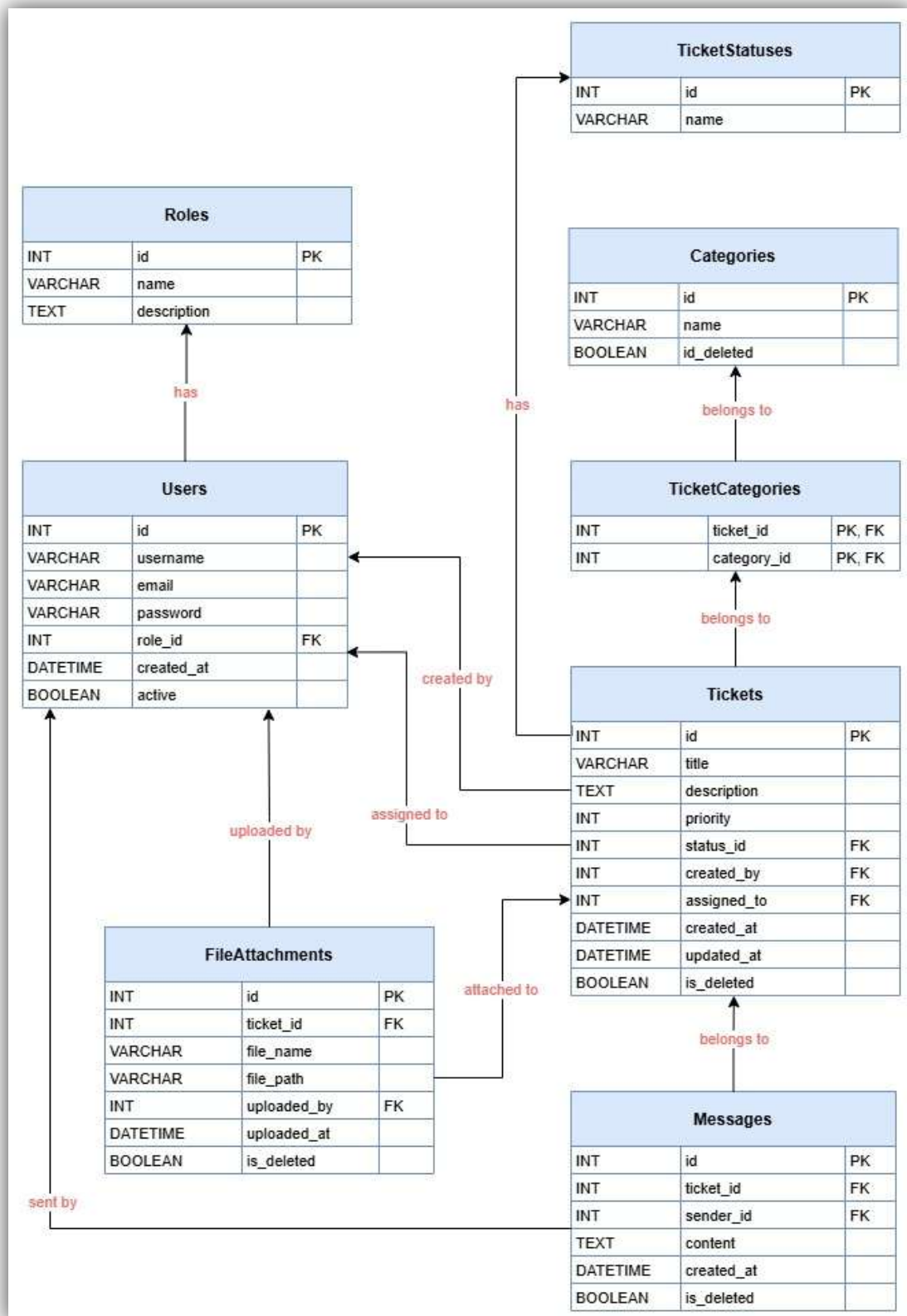
4.1.1. Diagrama E/R (entidad/relación)

Fig. 4-1-Diagrama E/R



4.1.2. Modelo relacional

Fig. 4-2-Modelo relacional



4.1.3. Patrones de diseño del software

Se han aplicado diferentes patrones y técnicas de arquitectura para estructurar el código y separar responsabilidades, para facilitar tanto su mantenimiento como su escalabilidad.

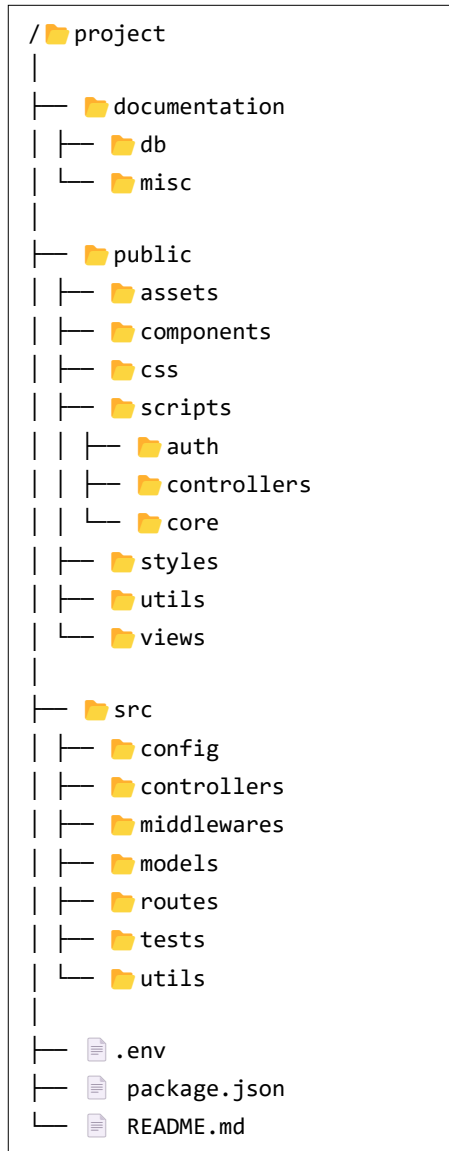
4.1.3.1 Justificación de patrones utilizados

- ✓ Se sigue la arquitectura **Cliente-Servidor**, separando claramente el frontend del backend, para facilitar el mantenimiento y la escalabilidad.
- ✓ El backend expone una **API REST**, lo que permite una comunicación estructurada entre cliente y servidor mediante peticiones HTTP.
- ✓ Para organizar la aplicación se ha utilizado el patrón **Modelo-Vista-Controlador** (MVC), separando el acceso a datos, la lógica de negocio y la presentación de la información.
Gracias a esta estructura, el código resulta más **mantenible, escalable y reutilizable**.
- ✓ La **comunicación** entre frontend y backend se realiza mediante **peticiones HTTP asíncronas** utilizando Fetch API de JavaScript. Esto permite actualizar dinámicamente la interfaz sin necesidad de recargar la página.
- ✓ El acceso a la API está protegido mediante **autenticación JWT** (JSON Web Token). Tras un inicio de sesión válido, el cliente almacena el token y lo envía en las peticiones a las rutas protegidas.
- ✓ Además se implementa un control de **acceso basado en roles**, permitiendo restringir funcionalidades según el tipo de usuario.

4.1.3.2 Estructura de la aplicación

Aquí se muestra cómo está organizado el proyecto en general, incluyendo la forma en que se estructuran las carpetas y algunos ejemplos de cómo una petición se procesa dentro de la aplicación.

Estructura de carpetas



Ejemplo de código para la creación de tickets

Modelo:

```
async function createTicket(title, description, priority, created_by) {
  const sql = `
    INSERT INTO Tickets (title, description, priority, created_by)
    VALUES (?, ?, ?, ?)
  `;
  const [result] = await db.execute(sql, [
    title,
    description,
    priority,
    created_by,
  ]);
  return result;
}
```

Controlador:

```
async function createTicket(req, res) {
  try {
    const { title, description, priority } = req.body;

    if (!title) {
      return res.status(400).json({ message: "Missing title" });
    }
    // Insert ticket in DB
    const result = await ticketModel.createTicket(
      title,
      description ?? null,
      priority ?? 5,
      req.user.id,
    );
    res.status(201).json({
      message: "Ticket created",
      data: {
        ticketId: result.insertId,
      },
    });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
}
```

Ruta:

```
// Any authenticated account can create new tickets
router.post(
  "/",
  authMiddleware,
  roleMiddleware([1, 2, 3]),
  ticketController.createTicket,
);
```

Vista:

```

async createTicket() {
  const modal = getController("modal").getInstance();
  const content = await (
    await fetch("../components/forms/ticket-create.form.html")
  ).text();

  modal.open({
    title: "New Ticket",
    content,
    footer: `
      <button class="btn btn-primary btn-wider-lg" data-
action="save">Save</button>
      <button class="btn btn-primary btn-wider-lg" data-
action="cancel">Cancel</button>
    `,
    actions: {
      save: async () => {
        const data = modal.getFormData();
        const title = data?.["modal-form-ticket-create-title"];
        const description = data?.["modal-ticket-create-description"];
        const priority = data?.["modal-ticket-create-priority-slider"];
        if (!title || !priority) {
          modal.close();
          return;
        }

        const token = localStorage.getItem("auth_token");
        const res = await fetch(`${API_BASE_URL}/tickets`, {
          method: "POST",
          headers: {
            "Content-Type": "application/json",
            Authorization: `Bearer ${token}`,
          },
          body: JSON.stringify({
            title,
            description,
            priority,
          }),
        });
        modal.close();
        if (!res.ok) {
          const error = await res.json();
          return;
        }

        await this.loadTickets();
        this.updateActiveList();
      },
      cancel: () => modal.close(),
    },
  });
}

```

Acceso y permisos

Cada cuenta tendrá un rol asignado, el cual determinará los accesos y permisos disponibles dentro de la aplicación.

Tabla 4-1: Roles y permisos

Rol	Permisos
Administrador	✓ Crear usuarios ✓ Modificar usuarios (excepto contraseña) ✓ Eliminar usuarios ✓ Crear tickets ✓ Modificar cualquier ticket (incluido cambio de estado) ✓ Eliminar cualquier ticket ✓ Crear categorías ✓ Modificar categorías ✓ Eliminar categorías ✓ Modificar su contraseña
Técnico	✓ Crear tickets ✓ Modificar tickets (incluido cambio de estado) ✓ Ver tickets creados por él mismo o asignados a él mismo ✓ Ver todos los tickets no asignados ✓ Asignarse tickets ✓ Desasignarse tickets asignados a él mismo ✓ Modificar su contraseña
Usuario	✓ Crear tickets ✓ Modificar tickets creados por él (incluido cambio de estado) ✓ Eliminar tickets creados por él ✓ Modificar su contraseña

Middlewares

Se introduce el uso de middlewares para controlar el acceso a las rutas antes de llegar a los controladores. Estos permiten realizar distintas comprobaciones:

- ✓ Verificar la autenticación del usuario mediante JWT.
- ✓ Comprobar si el token es válido o ha expirado.
- ✓ Validar los permisos del usuario según su rol.

Ejemplo de middleware

Definición:

```
function roleMiddleware(allowedRoles) {  
  return (req, res, next) => {  
    try {  
      if (!req.user) {  
        logger.warn("RoleMiddleware: authentication required");  
        return res.status(401).json({ message: "Authentication required" });  
      }  
  
      const userRoleId = getRoleId(req.user?.role);  
  
      if (!allowedRoles.includes(userRoleId)) {  
        logger.warn("RoleMiddleware: insufficient permissions");  
        return res.status(403).json({ message: "Insufficient permissions" });  
      }  
  
      // Finish this middleware and skips to next middleware or route  
      next();  
    } catch (error) {  
      logger.error(`RoleMiddleware: error checking permissions`);  
      return res.status(500).json({ message: error.message });  
    }  
  };  
}
```

Uso:

```
// Se limita el acceso únicamente para administrador y técnico  
router.get(  
  "/:id",  
  authMiddleware,  
  roleMiddleware([1, 2]),  
  userController.getUserById,  
);
```

4.1.4. Diseño de interfaces

4.1.4.1 Mapa del sitio

Al tratarse de una **Single Page Application** (SPA), la navegación se basa en rutas principales que cargan **distintas vistas** sobre un **único contenedor principal único**, sin necesidad de recargar la página. Las funcionalidades internas no generan nuevas rutas, sino que se gestionan mediante **componentes** dentro de cada vista principal, lo que permite una carga más rápida y un consumo de recursos menor.

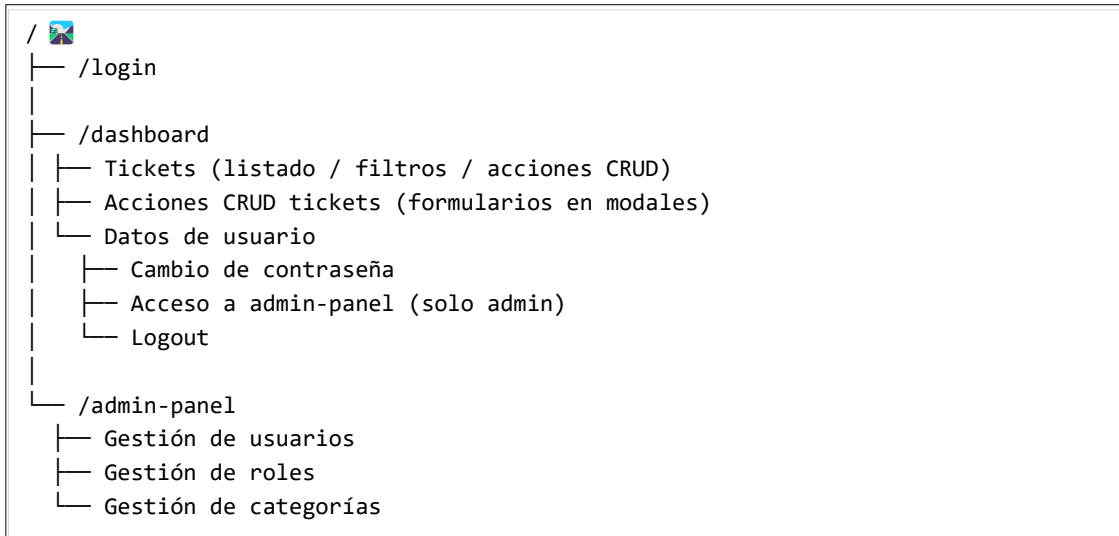
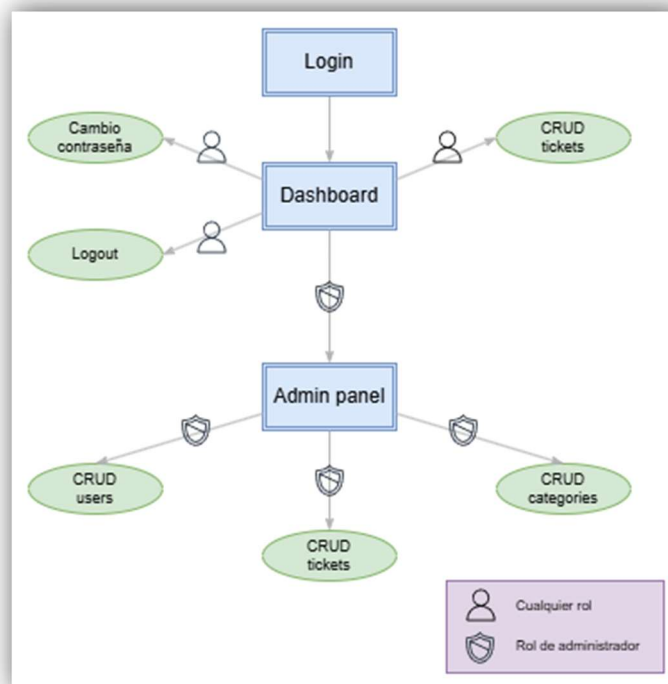


Fig. 4-3-Mapa del sitio



4.1.4.2 Guía de estilos

✓ **Tipografía:**

Se ha elegido Roboto, por ser una fuente moderna, neutra y de fácil lectura.

✓ **Paleta de colores:**

Se ha diseñado para buena legibilidad, con un estilo sobrio.

✓ **Jerarquía de tamaños de fuente:**

Se emplean distintos tamaños para jerarquía visual entre títulos y textos, para facilitar la lectura.

✓ **Otros elementos visuales (bordes, sombras, cards):**

Se utilizan bordes redondeados y sombras suaves para dar profundidad a la interfaz y resaltar mejor el contenido.

✓ **Estados interactivos (hover, active/selected, disabled):**

Se definen diferentes estados visuales para mejorar la interacción del usuario.

✓ **Diseño responsive:**

La interfaz se adapta a distintos tamaños de pantalla (móvil, tablet, escritorio).

```
// Background system
$bg: #f8fafc; // main background
$surface: #ffffff; // cards - containers
$surface-hover: #f1f5f9;
$surface-active: #e2e8f0; // active - selected
$surface-inactive: #ccd6e1;
$surface-dark: #c3c8ce; // darker variant
$overlay: rgba(15, 23, 42, 0.4); // modal backdrop

// Text
$text: #0f172a; // primary text
$text-muted: #475569; // secondary text - placeholders
$text-inverse: #edf1f5; // text on dark backgrounds

// Primary actions
$primary: #7a9de9;
$primary-hover: #eaf2fb;
$primary-active: #6f8ad5;

// Priorities & statuses
$success: #22c55e;
$success-dark: #16a34a;
$info: #3b82f6;
$info-dark: #2563eb;
$warning: #f59e0b;
$warning-dark: #d97706;
$error: #ef4444;
$error-dark: #dc2626;
$invalid: #6b7280;

// Font sizes
$size-xs: 0.5rem;
$size-sm: 0.8rem;
$size-md: 1.2rem;
$size-lg: 1.5rem;
$size-xl: 1.8rem;

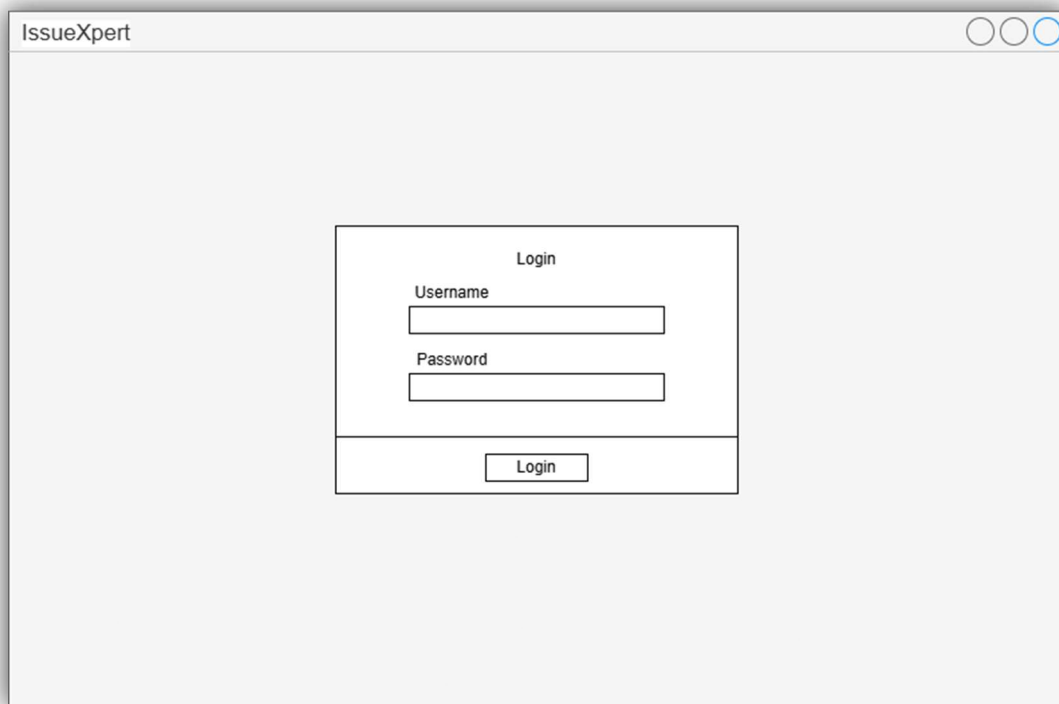
// Shadows
$shadow-sm: 0 2px 6px rgba(0, 0, 0, 0.15);
$shadow-md: 0 4px 10px rgba(0, 0, 0, 0.2);
$shadow-lg: 0 4px 14px rgba(0, 0, 0, 0.2);

// Radius
$radius-sm: 6px;
$radius-md: 10px;
$radius-lg: 14px;

// Spacing
$spacing-xs: 5px;
$spacing-sm: 10px;
$spacing-md: 16px;
$spacing-lg: 20px;
$spacing-xl: 24px;
```

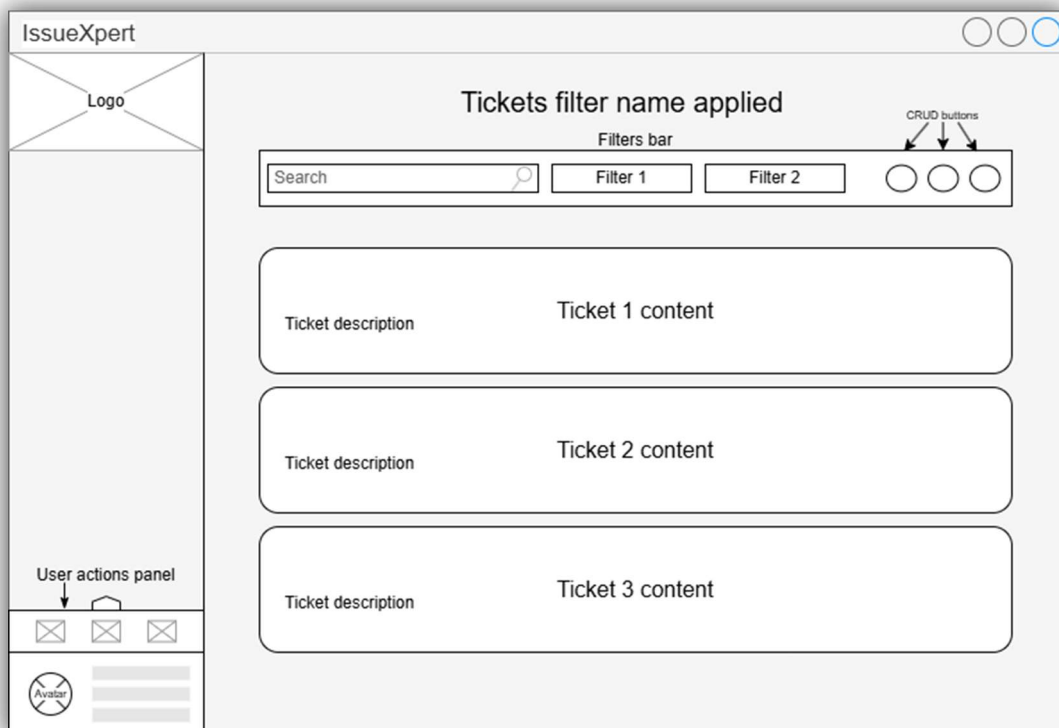
4.1.4.3 Bocetos

Fig. 4-4-Pantalla de login



The mockup shows a window titled "IssueXpert" with standard OS window controls. In the center is a "Login" form. The form has a title "Login", followed by a "Username" label and a text input field, then a "Password" label and a text input field. At the bottom of the form is a "Login" button.

Fig. 4-5-Pantalla del dashboard



The mockup shows a window titled "IssueXpert" with standard OS window controls. The dashboard layout includes:

- Left Sidebar:** Contains a "Logo" placeholder at the top, a large empty space in the middle, and a "User actions panel" at the bottom. The panel includes three square icons and an "Avatar" section with a circular icon and a name field.
- Main Content Area:**
 - Header: "Tickets filter name applied"
 - Filters bar: Contains a "Search" input field with a magnifying glass icon, and two buttons labeled "Filter 1" and "Filter 2". To the right of these buttons are three circular "CRUD buttons" (Create, Read, Update/Delete).
 - Ticket List: Three rounded rectangular cards stacked vertically. Each card contains a "Ticket description" label on the left and a ticket number (e.g., "Ticket 1 content", "Ticket 2 content", "Ticket 3 content") on the right.

Fig. 4-6-Pantalla del panel de administrador

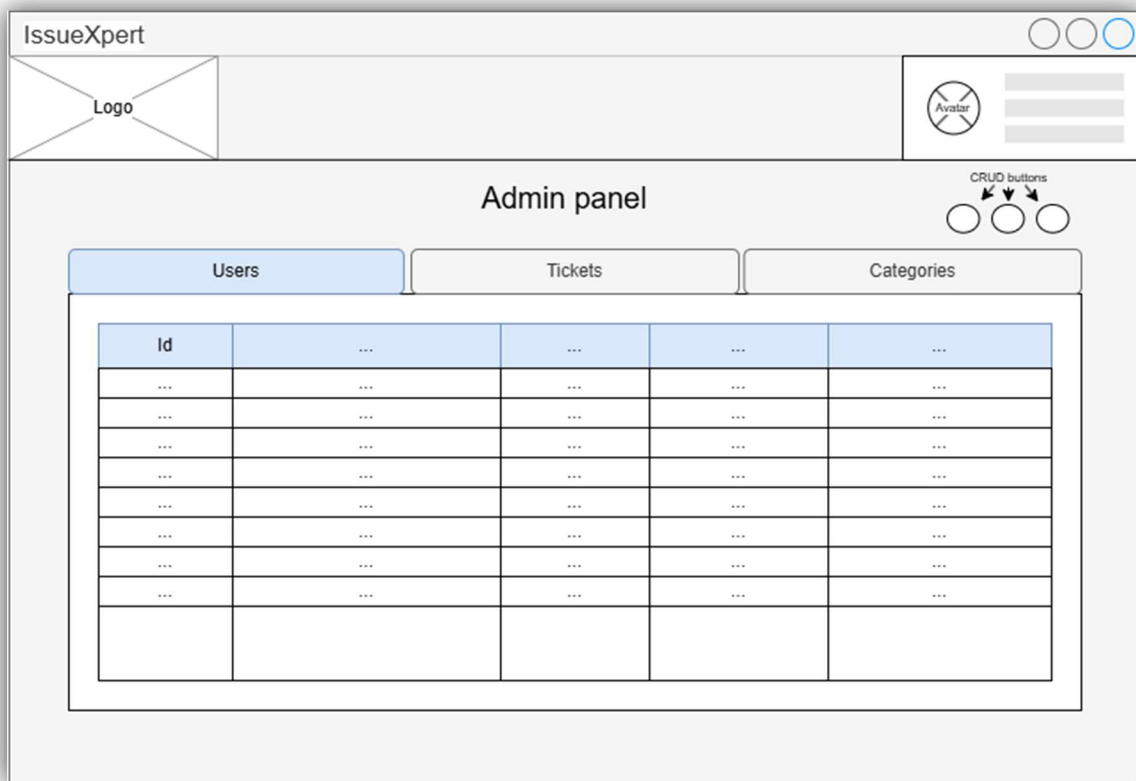
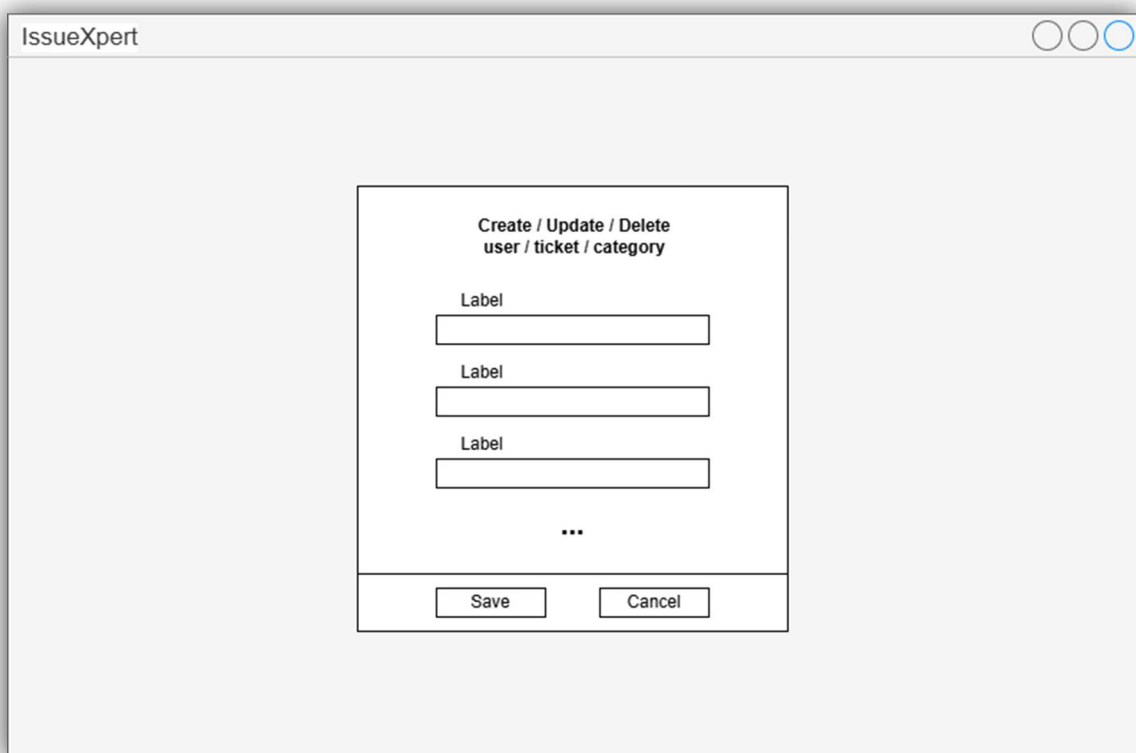


Fig. 4-7-Pantalla del modal (CRUD)



4.1.5. Arquitectura de frontend basada en componentes

Aunque el proyecto no utiliza frameworks como **React**, **Angular** o **Vue**, se ha intentado replicar algunos de sus conceptos fundamentales para mejorar la organización y escalabilidad del frontend.

En concreto, la interfaz se ha estructurado siguiendo una lógica basada en **componentes reutilizables**, separando las distintas partes de la aplicación en bloques independientes con su propia responsabilidad (vistas, modales, formularios...).

Además, se ha implementado una **gestión de estado** a nivel de aplicación para controlar la información mostrada en pantalla, permitiendo **actualizar dinámicamente** la interfaz en función de las acciones del usuario sin necesidad de recargar la página (filtros, pestañas activas...).

Este enfoque permite simular parte del comportamiento de frameworks modernos, facilitando el mantenimiento del código y mejorando la organización general del frontend, aunque sin depender de librerías externas.

4.1.6. Despliegue de la aplicación

La aplicación se ejecuta en una Raspberry Pi 5 que actúa como servidor de aplicaciones. Aquí se ejecutan la API desarrollada con Node.js y Express, y la base de datos MySQL que utiliza el sistema.

Para permitir el acceso a la aplicación desde Internet, se utiliza Cloudflare Tunnel. Esto permite publicar servicios internos de forma segura sin necesidad de abrir puertos en el router ni exponer la red local.

El dominio **hnezado.dev** se ha asociado al túnel de Cloudflare, lo que permite a los usuarios acceder a la aplicación mediante una dirección web personalizada.

La administración y mantenimiento del servidor se realizan de forma remota mediante conexión SSH, utilizando la herramienta MobaXterm para gestionar archivos y ejecutar comandos.

Esta arquitectura ofrece varias ventajas:

- ✓ Permite el acceso remoto seguro mediante HTTPS.
- ✓ No es necesario abrir puertos en el router.
- ✓ Ofrece protección adicional frente a accesos directos desde Internet.
- ✓ El coste de infraestructura es bajo gracias al uso de hardware propio.
- ✓ Y es fácil mantener y actualizar la aplicación.

Fig. 4-8-Estado del backend de IssueXpert en PM2

```
@ :~$ pm2 status
```

id	name	mode	u	status	cpu	memory
		fork	0	online	0%	94.1mb
		fork	0	online	0%	89.7mb
5	issuexpert-backend	fork	10	online	0%	69.3mb
		fork	18	online	0%	72.1mb

Fig. 4-9-Configuración del tunel de Cloudflare

```
GNU nano 7.2 .cloudflared/config.yml *
```

```
tunnel:
credentials-file:
ingress:
# issuexpert
- hostname: issuexpert.hnezado.dev
  service: http://localhost:80
```

Fig. 4-10-Configuración de Nginx para IssueXpert (reverse proxy)

```
GNU nano 7.2 /etc/nginx/sites-available/ *
```

```
# IssueXpert
server {
    listen 80;
    server_name issuexpert.hnezado.dev;

    location / {
        proxy_pass http://localhost:
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Estas configuraciones permiten el despliegue completo de IssueXpert en la Raspberry Pi 5, incluyendo la ejecución del backend mediante **PM2**, la publicación del servicio mediante **Cloudflare Tunnel** y la exposición del dominio a través de **Nginx** como reverse proxy, sin necesidad de abrir puertos en el router.

4.2. Pruebas

4.2.1. Pruebas funcionales

Tabla 4-2: Pruebas funcionales

ID	RF	Funcionalidad	Acción realizada	Resultado esperado	Resultado obtenido
P01	RF01	Inicio de sesión	Credenciales válidas	Acceso a la aplicación	Correcto
P02	RF01	Inicio de sesión	Credenciales incorrectas	Acceso denegado	Correcto
P03	RF12	Cambio de contraseña	Nueva contraseña válida	Contraseña actualizada	Correcto
P04	RF02	Crear usuario	Datos válidos	Usuario creado	Correcto
P05	RF02	Editar usuario	Modificación de datos	Usuario actualizado	Correcto
P06	RF02	Eliminar usuario	Eliminación lógica	Usuario eliminado	Correcto
P07	RF04	Crear ticket	Usuario autenticado	Ticket creado	Correcto
P08	RF05	Editar ticket	Datos válidos	Ticket actualizado	Correcto
P09	RF06	Eliminar ticket	Usuario autorizado	Ticket eliminado	Correcto
P10	RF07	Cambiar estado ticket	Técnico/Admin	Estado actualizado	Correcto
P11	RF08	Asignar ticket	Técnico se asigna ticket	Ticket asignado	Correcto
P12	RF09	Consulta de tickets	Según rol	Vista filtrada correcta	Correcto
P13	RF10	Filtrar tickets	Filtros aplicados	Resultados filtrados	Correcto
P14	RF11	Crear categoría	Datos válidos	Categoría creada	Correcto
P15	RF11	Editar categoría	Modificación	Categoría actualizada	Correcto
P16	RF11	Eliminar categoría	Eliminación lógica	Categoría eliminada	Correcto
P17	RF13	Acceso a panel admin	Usuario sin permisos	Acceso denegado	Correcto
P18	RF13	Acceso sin token	Sin JWT	Redirección a login	Correcto

*Todas las eliminaciones son lógicas, por lo que no se eliminan completamente, sino que se marcan como eliminadas.

4.2.2. Pruebas no funcionales

Tabla 4-3: Pruebas no funcionales

ID	Prueba	Descripción	Criterio de validación	Resultado
RNF01	Responsive	Uso en móvil, tablet y escritorio	Adaptación correcta de la interfaz sin desbordamientos ni pérdida de funcionalidad	Correcto
RNF02	Compatibilidad	Ejecución en Chrome y Brave	Funcionamiento correcto sin errores visuales ni funcionales	Correcto
RNF03	Seguridad de autenticación	Cifrado de contraseñas y acceso seguro	Contraseñas almacenadas cifradas y acceso mediante JWT válido	Correcto
RNF04	Gestión de sesión	Persistencia de sesión con JWT	El usuario mantiene sesión activa hasta expiración del token	Correcto
RNF05	Concurrencia	Acceso simultáneo de varios usuarios	Múltiples usuarios interactúan sin bloqueos ni errores	Correcto
RNF06	Rendimiento	Tiempo de carga de vistas y datos	Carga de vistas en tiempo razonable sin bloqueos visibles	Correcto

Metodología de pruebas

Todas las pruebas se hicieron de forma manual durante el desarrollo del proyecto.

Se verificó y comprobó el correcto funcionamiento de las distintas funcionalidades de la aplicación y se fueron corrigiendo sobre la marcha los errores que aparecían durante el proceso de prueba.

5. Conclusiones finales

El proyecto logró cumplir con los objetivos principales que se fijaron al principio. La aplicación permite hacer cosas básicas como autenticar a los usuarios, controlar el acceso según roles y gestionar usuarios, tickets y categorías de manera completa.

No se pudo asignar categorías a los tickets. Tampoco se pudo agregar el envío de mensajes dentro de los tickets o un chat en vivo. Estas cosas son más complicadas y se dejaron para después porque había que enfocarse en lo principal.

Aunque no está todo completo, el resultado final es bueno y cumple con lo que se quería lograr. La aplicación es funcional y se puede mejorar en el futuro.

5.1. Grado de cumplimiento de los requisitos fijados

- ✓ Cumplimiento de la estructura de la BD con 8 tablas.
- ✓ Sistema de autenticación (registro, login, roles).
- ✓ Permisos (administrador, técnico, usuario).
- ✓ Gestión de usuarios (CRUD).
- ✓ Gestión de tickets (CRUD).
- ✓ Sistema de estado de los tickets (abierto, en progreso, resuelto, cerrado).

5.2. Propuestas de mejora o ampliaciones futuras

- ✓ Añadir un spinner cuando los datos aún no se hayan cargado completamente para indicar al usuario estado de carga.
- ✓ Incorporar la asignación de una o más categorías a cada ticket.
- ✓ Cuando se hace scroll hacia abajo en la lista de los usuarios, tickets y categorías, dejar visibles permanentemente los botones de creación, edición y eliminación.
- ✓ Mensajería interna de cada ticket, para mejorar la comunicación entre el usuario y el técnico.
- ✓ Popup mensaje de alerta para dar información en tiempo real al usuario de cada acción, si se realiza con éxito, si hay algún error...
- ✓ Extraer la lógica de todas las vistas a componentes individuales y hacerlos más reutilizables.
- ✓ Historial de cambios en un ticket (cada modificación con su autor).
- ✓ Chat en tiempo real entre distintos usuarios.

6. Guías

6.1. Guía de administrador

El panel de administración solo está disponible para usuarios con el rol de administrador. Aquí se gestionan usuarios, tickets y categorías del sistema con operaciones completas de creación, lectura, actualización y eliminación.

La interfaz de este panel de administración se divide en tres secciones principales: usuarios, tickets y categorías. Se accede a ellas mediante pestañas. Cada sección muestra una tabla con los registros existentes disponibles. Se pueden seleccionar elementos o filas para su gestión.

Fig. 6-1-Pestaña de administración de usuarios

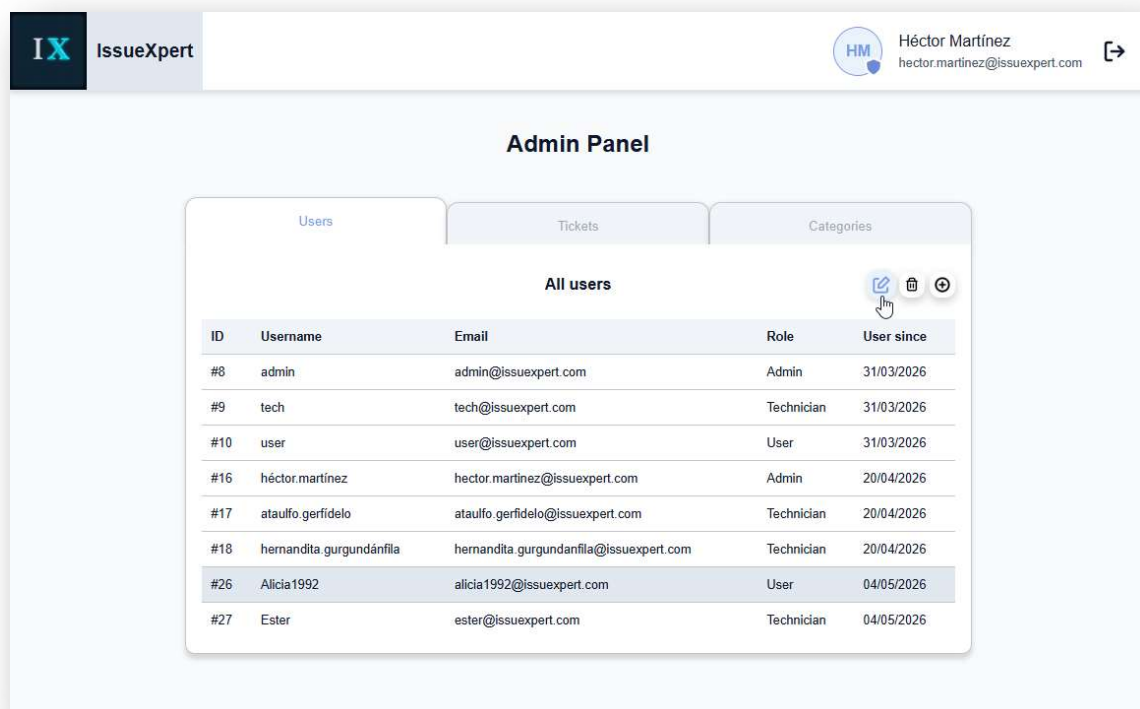
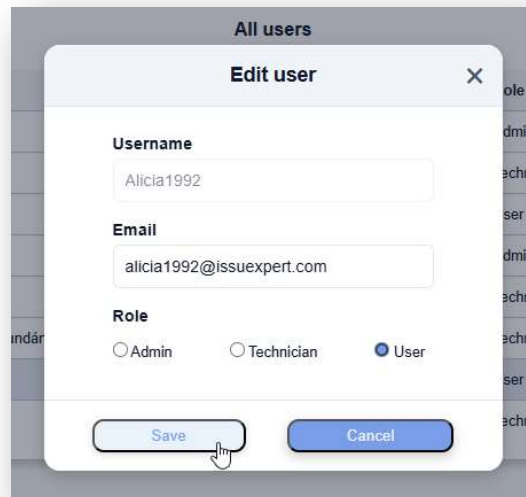


Fig. 6-2-Botones de acción (edición, eliminación y creación)



Las operaciones de creación, edición y eliminación se realizan a través de ventanas modales. Para editar o eliminar, primero se debe seleccionar un registro. La creación de nuevos registros está disponible directamente sin necesidad de seleccionar nada.

Fig. 6-3-Modal de edición de usuario

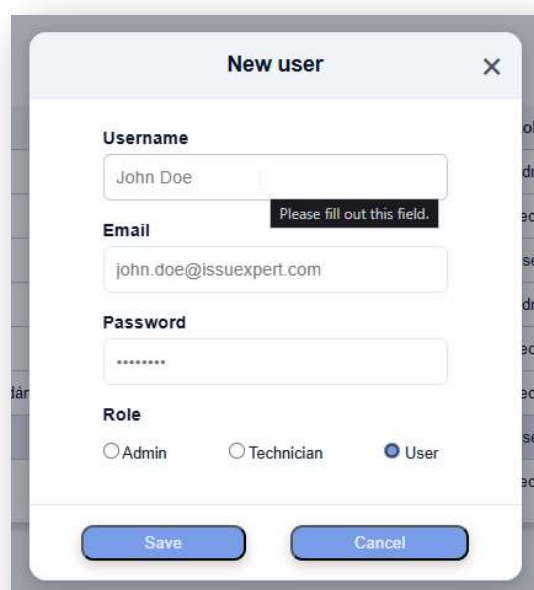


The image shows a modal window titled "Edit user" with a close button (X) in the top right corner. The modal is overlaid on a background window titled "All users". The "Edit user" modal contains the following fields and controls:

- Username:** A text input field containing "Alicia1992".
- Email:** A text input field containing "alicia1992@issuexpert.com".
- Role:** Three radio buttons labeled "Admin", "Technician", and "User". The "User" radio button is selected.
- Buttons:** Two buttons at the bottom: "Save" (light blue) and "Cancel" (blue). A mouse cursor is pointing at the "Save" button.

Este panel también permite crear nuevos usuarios. El registro no está disponible públicamente en la aplicación.

Fig. 6-4-Modal de creación de usuario



The image shows a modal window titled "New user" with a close button (X) in the top right corner. The modal is overlaid on a background window titled "All users". The "New user" modal contains the following fields and controls:

- Username:** A text input field containing "John Doe". A tooltip message "Please fill out this field." is visible next to the field.
- Email:** A text input field containing "john.doe@issuexpert.com".
- Password:** A text input field containing "*****".
- Role:** Three radio buttons labeled "Admin", "Technician", and "User". The "User" radio button is selected.
- Buttons:** Two buttons at the bottom: "Save" (blue) and "Cancel" (blue).

En la parte superior derecha, se muestra el perfil del usuario conectado. Desde allí, se puede cerrar la sesión.

Fig. 6-5-Perfil de usuario

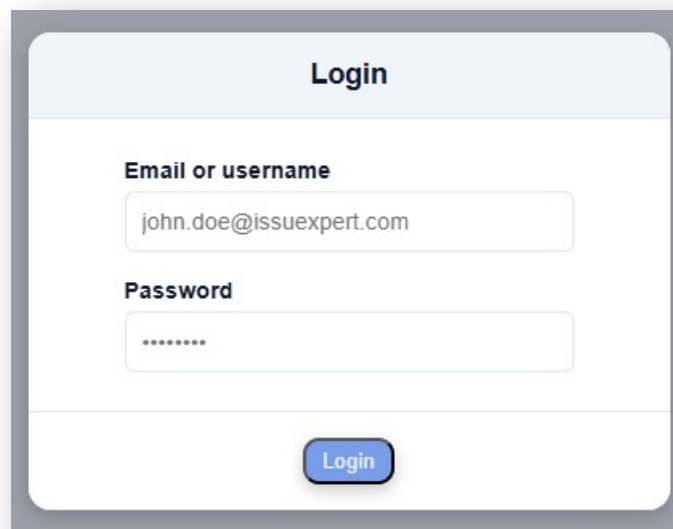


6.2. Guía de usuario

6.2.1. Acceso a la aplicación

Para acceder a la aplicación es necesario disponer de unas credenciales proporcionadas por un administrador. El usuario deberá introducir su nombre de usuario o su correo electrónico y contraseña en la pantalla de inicio de sesión.

Fig. 6-6-Pantalla de inicio de sesión



The screenshot shows a login window titled "Login". It contains two input fields: "Email or username" with the value "john.doe@issuexpert.com" and "Password" with masked characters "*****". A blue "Login" button is positioned at the bottom center of the form.

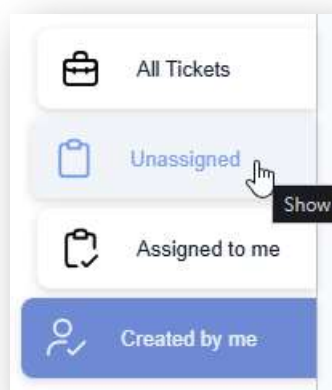
6.2.2. Panel principal (Dashboard)

Tras iniciar sesión correctamente, se accede al panel principal de la aplicación. En esta vista se muestran los tickets disponibles según los permisos del usuario autenticado.

Dependiendo del rol, el usuario podrá visualizar:

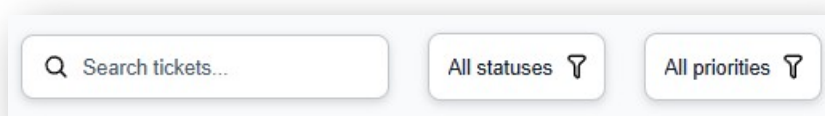
- ✓ Tickets creados por él.
- ✓ Tickets asignados a él.
- ✓ Tickets sin asignar.
- ✓ Vista combinada de tickets.

Fig. 6-7-Diferentes vistas de los tickets



Además, se dispone de filtros para localizar incidencias por texto, estado o prioridad.

Fig. 6-8-Barra de filtros



6.2.3. Gestión de tickets

Los usuarios pueden editar, eliminar y crear nuevos tickets mediante los botones de acción.

Fig. 6-9-Botones de acción sobre tickets (edición, eliminación y creación)

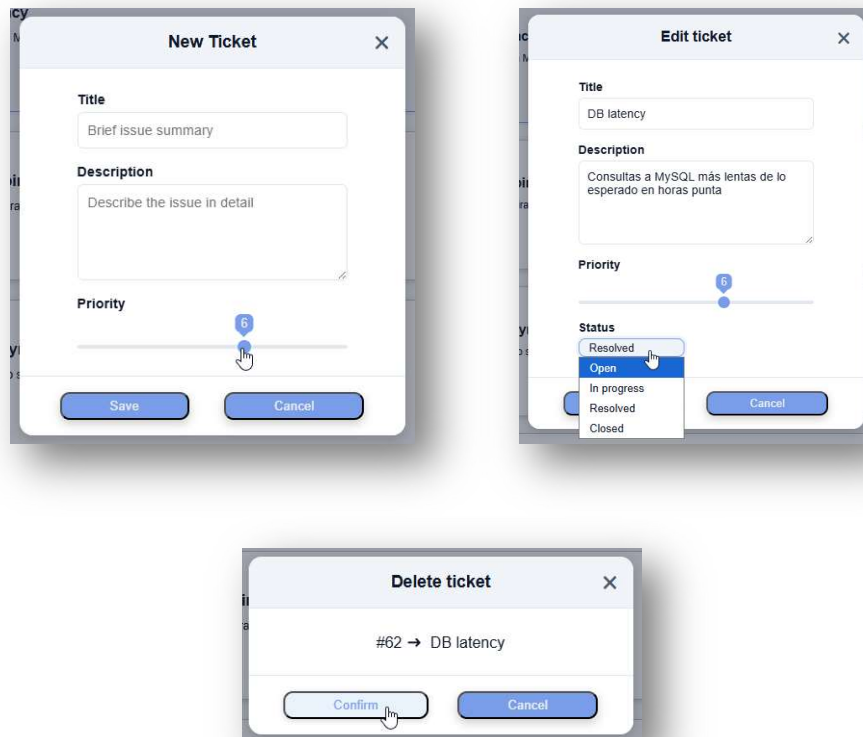


Cada ticket incluye información como:

- ✓ Título
- ✓ Descripción
- ✓ Prioridad
- ✓ Estado (solo accesible desde la edición)
- ✓ Usuario asignado (solo accesible desde el botón de asignación/desasignación)

Los tickets seleccionados pueden verse, editarse o eliminarse según los permisos asociados al rol del usuario.

Fig. 6-10-Modales de creación, edición y eliminación de ticket



6.2.4. Asignación y seguimiento

Los usuarios con permisos de técnico o administrador pueden asignarse tickets disponibles y actualizar su estado durante el proceso de resolución.

Los cambios realizados quedan reflejados inmediatamente en la interfaz.

Fig. 6-11-Ticket asignado y desasignado

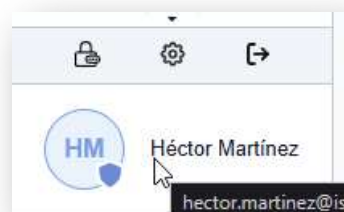


6.2.5. Perfil de usuario

Desde el perfil de usuario es posible consultar la información básica de la cuenta, incluyendo el nombre de usuario y su avatar (iniciales e icono del rol asignado). Al hacer hover sobre el avatar se muestra el nombre y el rol del usuario, y al hacerlo sobre el nombre de usuario se muestra su correo electrónico.

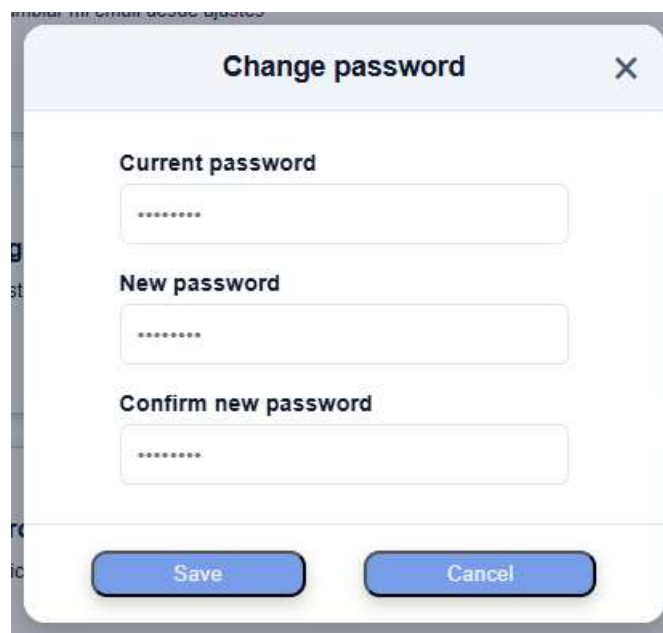
En la parte superior se encuentra el panel de acciones de usuario desplegable, desde el cual se puede cambiar la contraseña, acceder al panel de administración (solo usuarios administradores) o cerrar la sesión activa.

Fig. 6-12- Información de usuario con el panel de acciones de usuario



Al hacer clic en “Cambiar contraseña” (icono del candado), se abrirá un modal que solicitará la contraseña actual y la nueva contraseña dos veces para su confirmación.

Fig. 6-13-Modal de cambio de contraseña

A screenshot of a 'Change password' modal dialog. The modal has a title bar with the text 'Change password' and a close button (X). Inside, there are three input fields: 'Current password', 'New password', and 'Confirm new password'. Each field contains a series of dots representing masked text. At the bottom of the modal, there are two buttons: 'Save' and 'Cancel'.

7. Referencias

[1]	[En línea]. Consulta fetch Express con Node.js: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch (Mozilla.org) [Último acceso: 02 Marzo 2026]
[2]	[En línea]. Consulta mysql2 para conectar api con BBDD: https://ryan-m-schleck.medium.com/using-mysql-in-express-57f900f5e362 (Ryan M. Schleck) [Último acceso: 02 Marzo 2026]
[3]	[En línea]. Consulta JWT: https://stackoverflow.com/questions/39909419/what-are-the-main-differences-between-jwt-and-oauth-authentication (Joel Coehoorn) [Último acceso: 21 Marzo 2026]
[4]	[En línea]. Consulta mixins de Sass: https://sass-lang.com/documentation/at-rules/mixin [último acceso: 23 Abril]
[5]	[En línea]. Obtención de vectores e iconos: https://www.svgrepo.com/collection/kalai-oval-interface-icons [Último acceso: 20 Mayo 2026].
[6]	[En línea]. Consulta ventajas/desventajas de SPA, uso de componentes, estructura de proyecto, estilos y correcciones de código: https://chatgpt.com/share/6a1df6be-3250-83eb-85d4-40c372a5d954 [Último acceso: 25 Mayo 2026].